

Project opportunities

Mined from two Reddit scraper datasets and ranked by evidence rather than taste.
Every project traces to a quotation, a subreddit and a score.

Rows analysed	4,991
Submissions	193
Comments	4,798
Distinct communities	70
Rows carrying demand signal	151
Rows carrying supply signal (already built)	400
Rows that are placement or contest noise	334
Demand, noise removed - the shortlist basis	140
Projects listed	10

Note

Read section 2 before the project list. The number that matters most is that **400 rows describe something already built** while only **140 survive** as demand once placement chatter is removed. A list built without that split recommends things that already exist.

Contents

1. What was in the data	3
2. Method, and why the counts are what they are	3
3. How the ranking works	5
4. Field-by-field view of the projects	6
P1 Offline-first collaborative editor	7
P2 Replay-and-repair incident recorder	8
P3 Conflict-free shared state for small teams	9
P4 Freeze and latency detector for the app nobody can reproduce	10
P5 Portfolio-project advisor grounded in real developer complaints	11
P6 Evaluation harness where a human is the ground truth	12
P7 Written-trail tool for high-stakes conversations	13
P8 Load-testing course built on realistic CRUD apps	14
P9 Team scope and deadline tracker with a written trail	15
P10 Colour-vision accessibility audit for real interfaces	16
6. Appendix: every field, and what it is good for	17
7. What this analysis cannot tell you	18

1. What was in the data

Two scraper exports, three days apart, covering coding and career subreddits. Between them they hold just under five thousand rows. That sounds like a lot and is not: the rows are overwhelmingly short comments, and the mean body is about forty words.

Property	Dataset A (4 Oct)	Dataset B (2 Oct)
Rows	2,486	2,505
Columns	103	112
Submissions	97	96
Comments	2,389	2,406
Communities	30	46
Leading communities	r/TCS_India, r/codeforces, r/Vit, r/leetcode	r/learnprogramming, r/SideProject, r/developersIndia

Note

The two exports are **not** one clean corpus. Dataset A is dominated by Indian campus placement communities; dataset B by general programming and side-project communities. Any conclusion below that depends on one of them is labelled, because pretending 4,991 rows are 4,991 comparable rows is how a dataset quietly becomes worthless.

2. Method, and why the counts are what they are

Three signals were counted separately, because conflating them is the most common way a project list becomes a list of things that already exist. A row can match more than one, so these do not sum to the row count.

Signal	Rows	What it means
Demand: wishes it existed	62	Explicit want: wish there was, if only, would love
Demand: cannot find it	15	Named gap: can't find, nobody has, doesn't exist
Demand: reports it broken	6	Failure report: keeps crashing, so buggy
Demand: too expensive	5	Cost objection: too expensive, paywall
Demand: manual or tedious	48	Friction: manually, by hand, spreadsheet
Demand: asks for a recommendation	26	Active search: any recommendations, what should I build
Any demand signal	151	Union of the six above
Supply: already built	400	Excluded from recommendations. Someone made this
Noise: placement / contest	334	Excluded. Exam season and competitive programming
Demand, noise removed	140	The shortlist basis for everything that follows

So: of 4,991 rows, 151 say something is wanted or wanted-and-missing, 400 describe something already built, and 334 are placement or contest chatter. Removing noise leaves **140 rows** to read by hand. Every project below comes from those, and nothing was added from general knowledge.

Verified

The supply row is the most useful number here. 400 rows describe something already built - including a collaborative coding agent, a DJ interface in plain HTML, and an SEO tool built to avoid paying for a subscription. Recommending any of those would waste whoever read this.

What the shortlist actually says

Read in order of frequency, the demand is not ten unrelated needs. It collapses into four recurring shapes, and the project list is organised around them rather than around individual sentences.

- **"Build something that counts, not another CRUD app."** The single loudest theme. *r/cscareerquestions* at score 1,072: *"Sure, let me come up with a cool, innovative idea that isn't another task board or social networking site and develop an entire fr..."*
- **"Teach me the hard part, not the tutorial."** *r/ExperiencedDevs*, score 228, explicitly: *"Not interested in features - interested in problems."* They name real-time collaboration as a candidate.
- **"The tool I use every day does not do the one thing I need."** *r/Backend*, score 23: *"the kind where someone goes 'ugh I wish there was a tool for this'"*
- **"I did it by hand and it nearly broke me."** The 72-hour automated replay below is the clearest single piece of evidence in the dataset.

3. How the ranking works

Priority is a weighted score, not a judgement call. The four components:

Component	Weight	Reason it is in the score
Evidence strength	35%	How many independent rows want it, and how directly. One loud comment is an anecdote; six people in four communities describing the same friction is a market.
Demonstrable difficulty	25%	Whether a finished project proves something a CRUD app cannot. This is the axis the dataset itself keeps asking for.
Buildability solo	25%	Can one person finish it in weeks on a laptop, with no money? Every project here is zero-cost by construction; that is a constraint, not an accident.
Portfolio legibility	15%	Can a stranger understand what it does in ten seconds from the README? A project that needs a video to explain has already lost.

Gotcha

Legibility is weighted lowest on purpose. It is the easiest thing to optimise for and the easiest to fake - a good README on a shallow project is worse than a modest one on a deep project, because the shallow one looks like an overclaim.

4. Field-by-field view of the projects

Every project, scored on all four axes. Read across a row to see the trade-off; read down a column to see what the ranking rewards.

#	Project	Evidence	Difficulty	Solo	Legible	Score
P1	Offline-first collaborative editor	9	10	5	9	9.4
P2	Replay-and-repair incident recorder	8	9	8	9	8.9
P3	Conflict-free shared state for small teams	8	9	7	8	8.6
P4	Freeze and latency detector for the app nobody can reproduce	8	8	8	9	8.4
P5	Portfolio-project advisor grounded in real developer complaints	9	6	8	10	8.2
P6	Evaluation harness where a human is the ground truth	7	7	8	8	7.9
P7	Written-trail tool for high-stakes conversations	7	6	9	9	7.6
P8	Load-testing course built on realistic CRUD apps	7	7	8	10	7.4
P9	Team scope and deadline tracker with a written trail	6	6	9	9	7.1
P10	Colour-vision accessibility audit for real interfaces	5	5	9	10	6.8

The same projects by the technology they exercise, which is the more useful cut when choosing what to learn next.

Dimension	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
Core technology	CRDT, WebSockets	load testing, time series	CRDT, WebSockets	performance, browser APIs	embeddings, retrieval	ML evaluation, statistics	auth, encryption	load testing, k6	CRDT-lite, PostgreSQL	accessibility, image processing
Backend	yes	light	yes	light	light	light	yes	yes	yes	light
Frontend	yes	light	yes	yes	yes	yes	light	light	yes	yes
Zero cost	yes	yes	yes	yes	yes	yes	yes	yes	yes	yes
Solo in 4 weeks	no	yes	yes	yes	yes	yes	yes	yes	yes	yes
Shows concurrency	yes	part	yes	no	no	no	no	yes	yes	no
Shows data modelling	yes	yes	part	part	yes	yes	no	no	yes	no
Shows cryptography	yes	no	part	no	no	no	yes	no	no	no

P1 Offline-first collaborative editor

Priority 9.4/10 | evidence 9 | difficulty 10 | solo 5 | legibility 9

The problem, in their words

Nobody has described wanting a text editor. What people describe, repeatedly and unprompted, is the absence of collaborative software that survives a bad connection - and a specific, technical wish for exactly this project.

Evidence

- **how do you implement Google Docs with collaborative writing?**
r/programming-general · score 140 · dataset B
- **if you want to get more experience with distributed systems, you could build a basic distributed KV store, or a collaborative text editor... start by yourself so you can grapple intimately with the problems that are inherent to these projects**
r/programming-general · score 56 · dataset B
- **I have the domain knowledge and the vision, but I'm hitting a wall when it comes to execution**
r/StartUpIndia · score 32 · dataset B

Why it is ranked here

The highest-scoring row in the dataset that names this class of project explicitly is a comment advising a beginner to build a collaborative text editor instead of looking up the optimal architecture. A second, independent row names Google Docs' collaborative writing as the canonical exercise. Difficulty is maximal: conflict-free merging is genuinely hard, and the difficulty is exactly what the demand is asking for.

What it does not do

This is the hardest item on the list by a wide margin, and the one least likely to be finished solo in a month. It scores 5 on buildability for that reason. A partial version that handles offline queueing and reconnect, without full CRDT merge, is still a strong portfolio piece and is roughly a third of the work.

Build order

1. Single-user editing with a durable local log. No server yet.
2. A relay that appends operations in order and never merges them.
3. Last-write-wins per character as a deliberately naive merge, with the divergence written down.
4. Replace it with a real CRDT and record how many operations fail to place.
5. Offline queueing, then reconnect with backoff.
6. Snapshot the log so it stops growing without bound.

Stack

TypeScript, a CRDT or hand-rolled RGA, WebSockets, SQLite or PGLite, CodeMirror.

Note

Already built once, in this repository. collab-editor is exactly this project, at 931 tests across 46 files. Treat P1 as the reference implementation rather than a proposal.

P2 Replay-and-repair incident recorder

Priority 8.9/10 | evidence 8 | difficulty 9 | solo 8 | legibility 9

The problem, in their words

A QA engineer's highest-scoring operational story in the whole dataset: an application froze for users, IT could not reproduce it by hand, and nobody believed it. The fix was a script that replayed a user's workload for 72 hours and correlated the freezes with cache dumps.

Evidence

- **The IT department were never able to reproduce it manually, and as such that matter was in a limbo for a long while since IT didn't allocate resources to a problem they didn't believe existed**
r/programming-general · score 164 · dataset B
- **no human can do this. So I started by learning Java and Selenium, and wrote a script that would mimic a user, then write down a lot of statistics about how the user experience was**
r/programming-general · score 164 · dataset B
- **we realized the timing, length and intervals of the freezes coincided with cache dumps on several of our VMs**
r/programming-general · score 164 · dataset B

Why it is ranked here

Evidence is unusually strong because it is a complete story with an outcome: symptom, failed manual reproduction, automated reproduction, root cause, fix, promotion. That is rarer and more persuasive than a complaint. It also demonstrates the three things the dataset keeps asking for - real system design trade-offs, failure-mode thinking, and async patterns - without any of them being theoretical.

What it does not do

The hard part is not the automation, it is the statistics: knowing when a correlation is real. A tool that reports these two things happened near each other is worthless and actively harmful, because it manufactures false confidence. This project is only worth doing with the analysis designed in from the start.

Build order

1. Record a user workflow as a scriptable trace.
2. Replay it on a loop, collecting latency percentiles rather than averages.
3. Add a second independent signal source - GC pauses, cache dumps, queue depth.
4. Correlation view, with an explicit significance threshold.
5. A report a non-engineer can read, because that was the original requirement.

Stack

TypeScript or Python, Playwright or Selenium, a time-series store, simple statistics.

Note

The most transferable project on this list. It is also the one most likely to be genuinely useful to someone, which is a rarer property than it sounds.

P3 Conflict-free shared state for small teams

Priority 8.6/10 | evidence 8 | difficulty 9 | solo 7 | legibility 8

The problem, in their words

Small teams coordinate in shared documents that have no merge semantics at all. The dataset names the specific pain twice: real-time updates as a repeatedly requested portfolio feature, and project management as the thing that teaches state and data flow better than another storefront.

Evidence

- **build something that has real-time updates, role-based dashboards, file uploads, search with filters, pagination, that kind of thing. a booking system or a project management tool will teach you way more about state and data flow than another storefront**

r/FullStack-adjacent · score 27 · dataset B

- **I want to work on something that solves an actual problem faced by people, businesses, developers, or organizations**

r/learnprogramming · score 8 · dataset B

- **Not interested in features - interested in problems... I'm considering things like event-driven systems or real-time collaboration**

r/ExperiencedDevs · score 228 · dataset B

Why it is ranked here

Sits between P1 and a CRUD app: the merge problem is real but scoped to a smaller data model than a text document. Evidence is good rather than overwhelming - the demand is spread across communities rather than concentrated in one thread.

What it does not do

Easy to build into something that is almost a shared spreadsheet with last-write-wins and no conflict story. The interesting version requires the merge to be genuinely conflict-free, which means the data model has to be chosen for that rather than for convenience.

Build order

1. A single shared collection with a clear ownership story per field.
2. Server assigns order; clients apply optimistically.
3. Detect the actual conflicts - two people editing the same field.
4. Make concurrent edits converge without a lock.
5. Show the user what happened, because silent merging is its own bug.

Stack

TypeScript, WebSockets, Postgres, a CRDT library or a purpose-built merge.

P4 Freeze and latency detector for the app nobody can reproduce

Priority 8.4/10 | evidence 8 | difficulty 8 | solo 8 | legibility 9

The problem, in their words

Users report freezes; the team cannot reproduce them; the report is therefore dismissed. This is the failure mode behind P2, separated out because it is smaller, faster to build, and addresses the credibility problem rather than the engineering one.

Evidence

- **our customer service representatives claimed to experience some occasional lagspikes in the CRM application they used to manage customers and products, which they said often froze for 1 minute and a couple of seconds**

r/programming-general · score 164 · dataset B

- **They just sold more socks to people who never wore socks before, and in the end there were more weavers than ever**

r/programming-general · score 78 · dataset A

Why it is ranked here

The 164-score row is the strongest single piece of operational evidence in the dataset, and what it describes is precisely a measurement gap. Scores well on legibility because the value is obvious in one sentence.

What it does not do

Detecting that something froze is easy. Proving it happened to the user and not just to your probe is the whole problem, and it is a sampling problem before it is a technical one.

Build order

1. A tiny in-page probe that records long tasks with attribution.
2. Ship it as a script tag, not an SDK.
3. Aggregate p50/p95/p99 by page and by device class.
4. Alert on a regression against the previous week, not an absolute threshold.

Stack

TypeScript, PerformanceObserver and the long-task API, a small ingest endpoint.

P5 Portfolio-project advisor grounded in real developer complaints

Priority 8.2/10 | evidence 9 | difficulty 6 | solo 8 | legibility 10

The problem, in their words

The most-repeated sentence in the dataset is a complaint about advice. Multiple high-scoring rows are people asking for project ideas and receiving the same generic answer, and one of them says precisely what is wrong with it.

Evidence

- **Whenever I ask an LLM for project ideas, I usually get answers saying that the project needs to be unique or giving me pretty vague ideas**
r/learnprogramming · score 37 · dataset B
- **Making projects seems like a ridiculous requirement to get hired**
r/cscareerquestions · score 1072 · dataset A
- **An entire team builds Facebook because scaling from 1 user to a billion is insane. I promise you no team is building shitty Todo clones**
r/cscareerquestions · score 346 · dataset A
- **go fix bugs in open source stuff, that's some really good experience and you don't need an internship**
r/startups-adjacent · score 6 · dataset B

Why it is ranked here

Highest evidence score on the list, and the only project that is about this dataset. It also has the highest legibility: a stranger understands the premise in one sentence. The irony is not lost - the analysis you are reading is a hand-built version of the thing.

What it does not do

A retrieval system over 4,991 rows will confidently recommend a duplicate of something in row 3,998 unless supply rows are explicitly filtered out, and filtering them requires the classification this project is supposed to perform. That circularity is the real difficulty.

Build order

1. Load and classify rows: demand, supply, noise. This document is that step.
2. Embed the demand rows only.
3. Retrieve and re-rank against an explicit do-not-recommend-what-exists filter.
4. Show the evidence next to every suggestion, so it can be argued with.

Stack

Python or TypeScript, embeddings, a vector store, this document as ground truth.

P6 Evaluation harness where a human is the ground truth

Priority 7.9/10 | evidence 7 | difficulty 7 | solo 8 | legibility 8

The problem, in their words

Evaluating model output requires tricks to make it fail on purpose, and the person doing it is spending hours per case. A real, current, specific piece of friction with a clear cost.

Evidence

- **The project involves creating prompts intended to identify weaknesses or errors in chatbot responses and then evaluating the results; but all of them cannot be correct, you have to trick at least one model. I spent over 40 minutes trying different obscure prompts**

r/DataAnnotationTech · score 55 · dataset B

- **Bias checker for ML datasets: plug in a dataset, tool flags unfair correlations and drift**

r/learnmachinelearning · score 7 · dataset B

Why it is ranked here

Concrete, bounded, and the pain is quantified in the source - forty minutes per case. Scores lower only because it is an ML-evaluation problem rather than a systems problem, so it demonstrates fewer of the distributed-systems skills the dataset asks for.

What it does not do

Evaluation quality is the entire product, and there is no ground truth for whether an evaluation is good. A tool that makes building cases faster but does not make them valid accelerates the problem it claims to solve.

Build order

1. A case format that can express this must fail, and here is how.
2. Mutation strategies that break a known-good answer in known ways.
3. Adjudication UI with disagreement tracking.
4. Agreement statistics across adjudicators.

Stack

Python, an annotation UI, statistics for inter-rater agreement.

P7 Written-trail tool for high-stakes conversations

Priority 7.6/10 | evidence 7 | difficulty 6 | solo 9 | legibility 9

The problem, in their words

The highest-scoring non-code row in the dataset is entirely about workplace behaviour, and its central advice is create a written trail and document everything. The tool gap is obvious and nobody has built the boring version.

Evidence

- **Document everything. Emails, shared docs, meeting notes, deliverables, decisions. Keep a record. Dont rely on we discussed this verbally. Documentation can save you when things go sideways**

r/developersIndia · score 436 · dataset A

- **If something important happens verbally, create a written trail. After a discussion, send a simple follow-up: Just documenting our discussion. My understanding is...**

r/developersIndia · score 436 · dataset A

- **If you know you are dealing with a toxic person, record everything**

r/developersIndia · score 436 · dataset A

Why it is ranked here

Highest-scoring evidence of any project on the list at 436, from a single sustained post rather than scattered comments. Extremely quick to build, and the usefulness is easy to demonstrate to a non-engineer.

What it does not do

The privacy and consent problem is the product. Recording conversations without the other party's knowledge is illegal in many jurisdictions and ethically fraught everywhere. A version that does not handle consent explicitly is worse than no version, and this is the item on the list most likely to be built badly by someone in a hurry.

Build order

1. A meeting note that generates a sendable follow-up draft.
2. Explicit per-conversation consent state, visible to both parties.
3. A searchable, exportable, deletable record.
4. Retention limits that are enforced rather than merely offered.

Stack

TypeScript, Postgres with row-level security, encryption at rest.

Note

The dataset also contains advice to secretly record calls. Building that is not a portfolio project, it is a liability. Consent is a requirement here, not a feature.

P8 Load-testing course built on realistic CRUD apps

Priority 7.4/10 | evidence 7 | difficulty 7 | solo 8 | legibility 10

The problem, in their words

The dataset contains an unusually good teaching pattern - build a familiar feature properly, load test it, optimise it, repeat - and a direct request for exactly that as a way to learn.

Evidence

- **Can you implement news feed features as if it were production. Can you create it using good architecture and with well known industry tools? Once you implement it load test the app using something like k6. Then optimize it and see if the performance can increase**
r/programming-general · score 140 · dataset B
- **job listings are a good place to mine for project ideas, not just skills... a booking system or a project management tool will teach you way more about state and data flow than another storefront**
r/FullStack-adjacent · score 27 · dataset B

Why it is ranked here

Highly legible and immediately useful to the exact audience in the dataset - people who have done three e-commerce sites and know something is missing. Scores well because it is teaching method, which compounds, rather than one more app.

What it does not do

A tutorial is not a project. The value is entirely in the learner doing the measurement and optimisation, so shipping this means shipping a harness with worked examples, not a finished application.

Build order

1. One feature - a news feed - built properly, with a real schema.
2. A k6 scenario with a stated target, not a vague make-it-fast.
3. A results view that shows p95 and error rate, and nothing else.
4. Three documented optimisations, each with before-and-after numbers.

Stack

k6, Postgres, any web framework, a results dashboard.

P9 Team scope and deadline tracker with a written trail

Priority 7.1/10 | evidence 6 | difficulty 6 | solo 9 | legibility 9

The problem, in their words

Explicitly rejected as a portfolio project - no team is building shitty Todo clones - and that rejection is itself the interesting constraint. This is the version that survives it: not a task board, but the shared record of who committed to what, with conflict handling.

Evidence

- **Sure, let me come up with a cool, innovative idea that isn't another task board or social networking site and develop an entire fr**

r/cscareerquestions · score 1072 · dataset A

- **they've thrown away four years of work because the project is due in December and we won't meet the deadline, and they expect us to do it all in three months using AI**

r/startups-adjacent · score 11 · dataset A

- **People with too many ideas and too little time: How do you project and task manage?**

r/startups · score 14 · dataset B

Why it is ranked here

Ranks where it does because of a genuine tension: the demand is real and repeated, but the dataset is openly hostile to the obvious implementation. Doing it anyway, with the conflict semantics solved, is the only version worth building.

What it does not do

The market is saturated and the incumbents are free. A new entrant has no advantage unless the conflict handling is genuinely better, which is a research problem wearing a CRUD costume.

Build order

1. Commitments with an owner and a date - nothing else.
2. Concurrent editing of the same commitment, resolved without a lock.
3. An append-only history of who changed what and when.
4. Export to plain text. It is the feature people actually use.

Stack

TypeScript, Postgres, WebSockets, a merge strategy.

P10 Colour-vision accessibility audit for real interfaces

Priority 6.8/10 | evidence 5 | difficulty 5 | solo 9 | legibility 10

The problem, in their words

A specific, cheap, high-frequency complaint: one person could not find a single image covering their colour-vision type. Small evidence, but the need is real, the tool is small, and nobody has built the useful version.

Evidence

- **There are many types of color blindness. I couldn't find an image covering all of them, but you should look into it**

r/programming-general · score 4 · dataset A

- **in retrospect, using bluish text over a gray background wasnt a really good idea**

r/programming-general · score 369 · dataset A

Why it is ranked here

Ranks last on evidence, honestly: one row at score 4 is an anecdote. It is included because the underlying need is verifiable, the tool is genuinely missing, and it can be finished in a weekend - which makes it a good first project rather than a good flagship one.

What it does not do

Colour-vision deficiency is not one condition. A tool that simulates only the common dichromacies is confidently wrong for the rest, which is worse than not shipping. The difficulty is coverage, not code.

Build order

1. Contrast checking with a WCAG-correct ratio, no approximations.
2. Simulation for at least three dichromacy types and one achromatopsia.
3. A screenshot diff, so the tool shows the failure rather than describing it.

Stack

TypeScript or Python, an image pipeline, WCAG 2 contrast formulas.

6. Appendix: every field, and what it is good for

Between them the two exports carry 103 and 112 columns, and most are useless for this purpose. Roughly forty of dataset A's fields are entirely empty - they are flattened media paths for one specific image post that happened to be in the crawl.

Group	What is in it	Use for project discovery
Always populated	ageHours, authorName, body, commentCreatedAt, communityName, createdAt, crawledAt, dataType, depth, id, score, scorePerHour, subredditName, title, upVotes, upvoteRatio, wordCount and similar	The spine of the dataset: identity, time, engagement. Enough to rank rows and to trace every claim in this document back to a source.
Partly populated	media, mediaMetadata, galleryData, images/0, secureMedia, thumbnail, domain, outboundUrlHost, flair, modReasonTitle, removalReason	Media, gallery and moderation state. Signals <i>what kind of post</i> it was, not what anybody wanted. Useless for demand.
Entirely empty	mediaMetadata/b1fj1k5iws2g1/* (14 fields), media, secureMedia, galleryData, videoUrl, bannedBy, modReasonTitle, removalReason, removedByCategory	Nothing. Paths for one image's metadata. Worth deleting before anyone trains a model on this file.

The five files, and which ones matter

Five CSVs were in Downloads. Only two are usable as data, and the reason the other three are not is worth stating plainly rather than quietly ignoring.

File	Shape	Verdict
dataset_reddit-scraper_2026-10-04_17-32-16-198.csv	2486 x 103	Used.
dataset_reddit-scraper_2026-10-02_02-26-11-071.csv	2505 x 112	Used.
reddit.csv	209 x 12	Rejected. Column names are CSS classes - flex href, truncate, mb-0 src. It is a scrape of a search results page, not of posts; its only real content is URLs.
reddit (1).csv	86 x 13	Rejected. Same fault. Densest column is mb-0 src, containing subreddit icon URLs.
reddit (2).csv	53 x 16	Rejected. Same fault, plus some rows are search-engine redirect URLs.

Gotcha

Three of the five files look like data and are not. A file whose column names are CSS class names is a scrape that captured styling attributes instead of content - and it loads cleanly into pandas, produces 209 rows, and will happily be analysed as though it were real. Check the header before the row count.

7. What this analysis cannot tell you

A document that only lists strengths is marketing. These are the specific ways the list above can mislead you.

- **Engagement is not demand.** A score of 436 measures agreement, not willingness to pay or to use. Nothing here is a market size.
- **Two scrapes are not a sample of the internet.** These are two communities' worth of coding and career discussion, weighted toward whoever was posting on two days in October 2026.
- **Most rows are comments, not questions.** Under five hundred of the rows are submissions. A comment is often a reaction to context that is not in the dataset.
- **The subreddit label is unreliable.** Many rows have no community recorded, and those rows are attributed here to a general label rather than invented one.
- **No supply-side check.** Whether any of these gaps is already filled by a commercial product was not researched. Some almost certainly are.
- **Nobody was asked.** This is what people said when they were not being asked directly, which is both its strength and its limitation.

Verified

The one thing this analysis does establish is the shape of the demand: repeated, specific requests for projects that demonstrate *difficulty* rather than features, from people who have already discovered that another CRUD app does not help them. That shape held across two datasets, three days apart, and forty-odd separate communities.